

Deliverable #2: The Programmer's Manual

Group #2 – Scrabble Game

Blake Bozarth
Nono Moss
Joseph Simpson
Wendy Stump

Introduction

This manual contains everything a software engineer needs if he or she were to help develop or maintain the software for this scrabble-based project. Enclosed are our requirements documentation, design documentation, source code, testing plans, and known bugs and issues.

Contents

1 Requirements Documentation	4
1.1 Introduction	4
1.1.2 Scope Statement	4
1.1.3 Definitions, Acronyms, & Abbreviations	5
1.1.4 References	7
1.2 General Description	8
1.2.1 Product Perspective	8
1.2.2 Product Functions	8
1.2.3 User Characteristics.....	8
1.2.4 General Constraints	8
1.2.5 Assumptions & Dependencies	8
1.3 Specific Requirements	9
2 Design Documentation	12
2.1 Architecture Diagram (UML)	12
2.2 Pseudocode	13
2.3 Decision Tables.....	17
2.4 Control Diagrams	18
2.4.1 Game State Transition	18
2.4.2 Turn Submission	19
2.4.6 Tile Placement.....	20
2.4.3 Rules Engine.....	21
2.4.4 Scoring	22
2.4.5 Timer	23
3 Source Code Documentation	24
GameController.java.....	24
RulesEngine.java	26
ScoreCalculator.java	31
EndConditionEvaluator.java	35
TimeManager.java.....	36

GameState.java	37
Board.java	39
Square.java	41
Tile.java	43
Rack.java	44
TileBag.java	46
PlacementRequest.java	48
PlacedTile.java	48
WordValidation.java	49
JavaFXMain.java	50
Main.java	59
4 Testing Documentation	60
5 Known Bugs & Issues	62

1 Requirements Documentation

1.1 Introduction

1.1.2 Scope Statement

Group 2 will be developing a digital Scrabble-style game as our semester project. The core game mode will be a single-player, time-based game in which the user takes successive turns attempting to reach a target score before time expires. The required score and time limit will be determined by the selected difficulty level (easy, medium, or hard). All word placement rules, tile distribution, and board scoring will follow the standard rules of traditional Scrabble. The player will be provided with seven randomly drawn letter tiles and will draw new tiles after each turn to maintain a rack of seven tiles. The automatic word validation will be based on the SOWPODS dictionary.

If time permits, the following features will be implemented in the order listed:

- A two-player local pass-and-play mode following traditional Scrabble rules.
- A boss battler mode that introduces additional rogue-like mechanics layered onto the time-based single-player mode.
- A two-player mode against a computer-controlled opponent utilizing basic artificial intelligence.

The game will be implemented in Java using a graphical user interface built with JavaFX. The final application will be packaged as an executable JAR file, allowing it to run on Windows, macOS, and Linux systems with a compatible Java Runtime Environment installed. The software will function as a standalone application and will not require any additional software beyond the Java runtime.

1.1.3 Definitions, Acronyms, & Abbreviations

Definitions

Automatic Word Validation – System-based verification that a word exists in the approved dictionary before scoring.

Bag – The pool containing all un-played tiles from which new tiles are drawn.

Bingo – A move in which all seven tiles are played in a single turn, awarding a point bonus.

Board – The 15 x 15 grid on which tiles are placed to form words.

Cross Word – A secondary word formed perpendicular to the main word placement.

Difficulty Level – Game setting (Easy, Medium, Hard) that determines time limit and required target score.

Exchange – Replacing one or more tiles with new ones from the bag.

Pass – A turn where no tiles are played.

Premium Square – A special board square that modifies score:

- Double Letter (DL)
- Triple Letter (TL)
- Double Word (DW)
- Triple Word (TW)

Rack – The holder containing up to seven tiles available to a player.

Rack Leave – Tiles remaining after a move.

Score Multiplier – The cumulative effect of premium squares applied during a turn.

Seeded Randomization – Pseudorandom tile generation method.

Target Score – The required number of points the player must reach before the timer expires.

Tile – A letter piece with an associated point value. A standard set contains 100 tiles including two blanks.

Time-Based Mode – Single-player mode where the player must reach a target score within a fixed time limit.

Turn – A single move in which a player places a word, exchanges tiles, or passes.

Acronyms & Abbreviations

CSW – Collins Scrabble Words; international word list.

NASPA – North American Scrabble Players Association; governs North American competitive Scrabble.

NWL – NASPA Word List; official tournament word list used in the United States and Canada.

SOWPODS – Former name for CSW; derived from combining UK and US word lists.

WESPA – World English-Language Scrabble Players Association; governs international competitive Scrabble.

DL – Double Letter.

DW – Double Word.

GUI – Graphical User Interface.

JAR – Java Archive file format used to distribute Java applications.

JRE – Java Runtime Environment.

OT – Overtime.

UI – User Interface.

TL – Triple Letter.

TW – Triple Word.

1.1.4 References

Scrabble Rules

Team, H. C. (n.d.). *SCRABBLE*. Hasbro Instructions. <https://instructions.hasbro.com/en-us/instruction/scrabble-board-game>

Wespa. (n.d.). *WESPA - World English Language Scrabble® Players Association*. <https://wespa.org/>

Word List

Scrabble/scrabble/sowpods.txt at master · jesstess/Scrabble. (n.d.). GitHub. <https://github.com/jesstess/Scrabble>

Tools

Java | Oracle. (n.d.). <https://www.java.com/en/>

JavaFX. (n.d.). <https://openjfx.io/>

Porter, B., Van Zyl, J., & Lamy, O. (n.d.). *Welcome to Apache Maven – Maven*. <https://maven.apache.org/>

ChatGPT. (n.d.). ChatGPT. <https://chatgpt.com/>

ChatGPT was used as a supplemental tool to assist with debugging, refining JavaFX and RulesEngine class structure, and improving adherence to project requirements. Specifically, it helped structure UI class implementation, provided guidance on implementing specific features included in the RulesEngine for word validation and scoring, and helped with .bat file creation.

1.2 General Description

1.2.1 Product Perspective

This software reimagines the traditional Scrabble experience as a time-based single-player game for desktop users. While Scrabble is typically played as a turn-based game with multiple players, this version shifts the focus to a solo challenge requiring quick thinking and strategic word placement under time constraints. The base application provides an engaging way to enjoy the core mechanics of Scrabble without requiring physical materials or additional players, making it a more flexible and accessible option for casual gaming.

1.2.2 Product Functions

The base game will display a scrabble board and seven-tile player rack through a GUI, allowing the player to place tiles to form words according to placement rules. The system will validate the words played against the dictionary and automatically calculate scoring using traditional / tournament Scrabble scoring with project-specific adjustments as needed. The base game is time-based: the player must reach a target score before the timer expires. The game ends when the player meets the win condition or when time runs out.

1.2.3 User Characteristics

The end users of the application are individuals who enjoy casual word and puzzle games, such as Scrabble or mobile games like Candy Crush. These end users are expected to have basic computer proficiency and familiarity with graphical interfaces, including clicking, dragging, and interacting with on-screen elements. No other specialized knowledge is required. The end users may range from college students to general desktop users.

1.2.4 General Constraints

1. The application will be built using Java and JavaFX.
2. It must run as a standalone executable JAR.
3. It must run on Windows and macOS with a compatible JRE installed.
4. The application will have no network or multiplayer features and will be local-only.
5. Development time is constrained by the project timeline given within the class.

1.2.5 Assumptions & Dependencies

1. Assumes access to the word list resource packaged with the application and is accessible at runtime.
2. Assumes a standard desktop operating environment capable of running Java applications.

1.3 Specific Requirements

1.3.1 Core Game State Requirements

- 1.3.1.0** The system shall maintain a single GameState object as the authoritative source of truth for all game data.
- 1.3.1.1** The GameState shall contain currentScore.
- 1.3.1.2** The GameState shall contain remainingTime.
- 1.3.1.3** The GameState shall contain targetScore.
- 1.3.1.4** The GameState shall contain boardState.
- 1.3.1.5** The GameState shall contain gameStatus with possible values: RUNNING, WON, LOST.
- 1.3.1.6** Only the RulesEngine and GameController shall modify GameState.
- 1.3.1.7** Invalid placement attempts shall not modify GameState.

1.3.2 Board and Square Requirements

- 1.3.2.0** The system shall initialize a 15x15 2D Board grid at game start.
- 1.3.2.1** The Board shall contain Square objects.
- 1.3.2.2** Each Square shall maintain row and column position.
- 1.3.2.3** Each Square shall maintain an occupied flag.
- 1.3.2.4** Each Square shall maintain a bonusValue.
- 1.3.2.5** A Square marked as occupied shall not accept additional tiles.
- 1.3.2.6** Bonus squares shall apply their effect only once when first used.

1.3.3 Tile Requirements

- 1.3.3.0** Each Tile shall contain a character.
- 1.3.3.1** Each Tile shall derive scoring from word length logic.
- 1.3.3.2** Tiles shall only be placed onto unoccupied Squares.
- 1.3.3.3** Tile placement shall occur only through validated PlacementRequest processing.

1.3.4 Word Placement and Validation Requirements

- 1.3.4.0** All moves shall be submitted through a PlacementRequest.
- 1.3.4.1** A PlacementRequest shall include a word string.
- 1.3.4.2** A PlacementRequest shall include a starting position.
- 1.3.4.3** A PlacementRequest shall include a direction (horizontal or vertical).
- 1.3.4.4** The RulesEngine shall validate that all tiles are within board boundaries (validateSquaresAvailable).
- 1.3.4.5** The RulesEngine shall validate that the move forms at least one valid word of length two or greater (validateMove).
- 1.3.4.6** The RulesEngine shall validate that all placed tiles are on unoccupied squares (validateSquaresAvailable).

1.3.4.7 The RulesEngine shall validate that all words formed exist in the dictionary (validateMove).

1.3.4.8 The RulesEngine shall validate all cross-words created by the placement (getAllWordsFormed + validateMove).

1.3.4.9 The RulesEngine shall construct all words formed from the placement for validation (getAllWordsFormed).

1.3.4.10 If validation fails, validateMove shall return false and no state mutation shall occur.

1.3.4.11 If validation succeeds, validateMove shall return true and the GameController shall proceed with scoring and state updates.

1.3.5 Scoring Requirements

1.3.5.0 All scoring shall be handled by ScoreCalculator.

1.3.5.1 The ScoreCalculator shall calculate the total score for a submitted turn using calculateTurnScore(GameState, List<PlacedTile>).

1.3.5.2 The ScoreCalculator shall identify all words formed by the move using getWordsFormed(Board, List<PlacedTile>).

1.3.5.3 The ScoreCalculator shall compute each word score by summing the point values of all tiles in that word.

1.3.5.4 The ScoreCalculator shall apply letter bonuses and word bonuses only to newly placed tiles on unused bonus squares.

1.3.5.5 The ScoreCalculator shall prevent duplicate word scoring by tracking already-scored word positions.

1.3.5.6 The ScoreCalculator shall add a 50-point bonus when all seven rack tiles are used in one turn.

1.3.5.7 The ScoreCalculator shall consume used bonus squares after scoring so bonuses apply only once.

1.3.5.8 The ScoreCalculator shall return the total score for the move.

1.3.5.9 Scoring logic shall be deterministic and independent of the UI layer.

1.3.6 Time System Requirements

1.3.6.0 The GameState shall initialize remainingTime to the initial time limit at game start.

1.3.6.1 The TimeManager shall decrement remaining time only when the game status is RUNNING (decrement).

1.3.6.2 The TimeManager shall update time after each turn using tickAfterTurn(GameState).

1.3.6.3 The TimeManager shall determine if time has expired using checkTimeExpired(GameState).

1.3.6.4 If time expires, the EndConditionEvaluator shall update the GameState to LOST.

1.3.7 End Condition Requirements

1.3.7.0 EndConditionEvaluator shall check win condition using checkWinCondition(score, targetScore).

- 1.3.7.1 A win shall occur when `currentScore >= targetScore`.
- 1.3.7.2 `EndConditionEvaluator` shall check lose condition using `checkLoseCondition(timeRemaining)`.
- 1.3.7.3 A loss shall occur when `remainingTime <= 0`.
- 1.3.7.4 Upon win, `GameState.gameStatus` shall be set to `WON`.
- 1.3.7.5 Upon loss, `GameState.gameStatus` shall be set to `LOST`.

1.3.8 Game Flow Control Requirements

- 1.3.8.0 All placement input shall be processed by `GameController`.
- 1.3.8.1 `GameController` shall call `RulesEngine` for validation.
- 1.3.8.2 Upon successful validation, `GameController` shall update `GameState`.
- 1.3.8.3 `GameController` shall trigger `ScoreCalculator` for scoring.
- 1.3.8.4 `GameController` shall trigger `TimeManager` updates as needed.
- 1.3.8.5 After each move, `GameController` shall invoke `EndConditionEvaluator`.

1.3.9 UI Layer Requirements

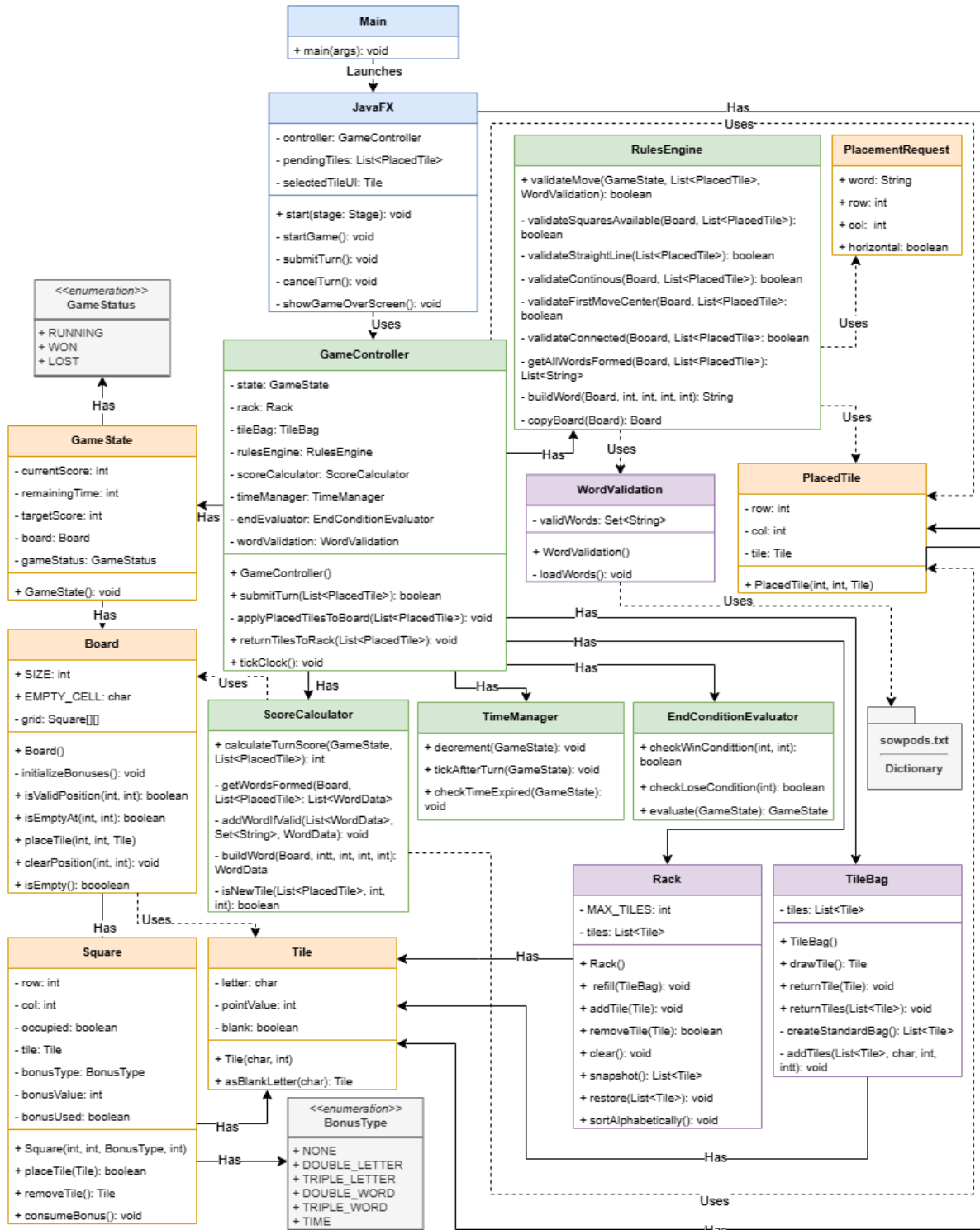
- 1.3.9.0 The JavaFX UI shall render the Board using `updateBoardUI`.
- 1.3.9.1 The JavaFX UI shall display `currentScore` using `scoreLabel` in `refreshUI`.
- 1.3.9.2 The JavaFX UI shall display `remainingTime` using `timerLabel` in `refreshUI`.
- 1.3.9.3 The JavaFX UI shall display game results on the end screen using `showGameOverScreen`.
- 1.3.9.4 The JavaFX UI shall collect tile input through drag-and-drop from the rack.
- 1.3.9.5 The JavaFX UI shall collect placement input by dropping tiles onto board squares.
- 1.3.9.6 The JavaFX UI shall store unsubmitted placements locally in `pendingTiles` until `Submit Turn` is pressed.
- 1.3.9.7 The JavaFX UI shall submit pending placements to `GameController` using `submitTurn`.
- 1.3.9.8 The JavaFX UI shall cancel unsubmitted placements using `cancelTurn` without changing the committed Board.
- 1.3.9.9 The UI layer shall not directly commit tiles to `GameState`; committed state changes shall occur through `GameController`.

1.3.10 Testing Surface Requirements

- 1.3.10.0 `RulesEngine` shall be testable independently of UI.
- 1.3.10.1 `ScoreCalculator` shall be testable independently.
- 1.3.10.2 `TimeManager` shall be testable independently.
- 1.3.10.3 `EndConditionEvaluator` shall be testable independently.
- 1.3.10.4 All validation failures shall produce deterministic results for test reproducibility.

2 Design Documentation

2.1 Architecture Diagram (UML)



2.2 Pseudocode

TimeManager.java

```

class TimeManager:
    private static int remainingSeconds = 600
    private static boolean running = false

    function start():
        running = true
    function tick():
        if running and remainingSeconds > 0:
            remainingSeconds -= 1
    function addSeconds(int delta):
        remainingSeconds += delta
    function getRemainingSeconds() -> int:
        return remainingSeconds

```

Specific Requirements: 1.3.6.1, 1.3.6.2, 1.3.6.3
 Stage Label: Clock System

PlacementsRequest.java

```

function buildPlacements(GameState S, PlacementRequest I) -> Placements:
    placements = empty list

    for r in 0..Board.SIZE-1:
        for c in 0..Board.SIZE-1:

            sTile = S.getSquare(r,c).getTile()
            iTile = I.getSquare(r,c).getTile()

            if sTile == EMPTY and iTile != EMPTY:
                placements.add(Placement(r,c,iTile))

            if sTile != EMPTY and iTile != sTile:
                // handled by predicate later
                continue

    return new Placements(placements)

```

Specific Requirements: 1.3.4.0, 1.3.4.1, 1.3.4.2, 1.3.4.3, 1.3.4.9
 Stage Label: Derived Data

WordValidation.java

```

function buildPrimaryWord(Placements P, GameState S) -> ValidatedWord:

    if P.size == 1:
        p = P.get(0)

        if hasNeighborLeftOrRight(p, S):
            axis = HORIZONTAL
        else:
            axis = VERTICAL
        else:
            if all rows equal:
                axis = HORIZONTAL
            else:
                axis = VERTICAL

    wordPlacements = empty list

    if axis == HORIZONTAL:
        row = P.get(0).row
        minCol = min col in P
        maxCol = max col in P

        // expand left
        c = minCol
        while c >= 0 and S.getSquare(row,c).hasTile():
            c--
        start = c + 1

        // expand right
        c = maxCol
        while c < SIZE and S.getSquare(row,c).hasTile():
            c++

        end = c - 1
        for col in start..end:
            tile = resolveTile(col,row,P,S)
            wordPlacements.add(Placement(row,col,tile))

    else:
        // same logic vertical

    return new ValidatedWord(wordPlacements)

function buildCrossWords(Placements P, GameState S) -> List<ValidatedWord>:
    crossWords = empty list
    for each placement p in P:
        if primary axis == HORIZONTAL:
            axis = VERTICAL
        else:
            axis = HORIZONTAL
        word = scan perpendicular from p

        if word.size >= 2:
            crossWords.add(word)
    return crossWords

```

Specific Requirements: 1.3.4.5, 1.3.4.7, 1.3.4.9, 1.3.4.8, 1.3.4.9
 Stage Label: Derived Data

ScoreCalculator.java

```
function computeScore(primaryWord, crossWords, S) -> int:
```

```
    primaryScore = computeWordScore(primaryWord, S)
```

```
    crossScore = 0
```

```
    for word in crossWords:
```

```
        crossScore += computeWordScore(word, S)
```

```
    return primaryScore + crossScore
```

```
function computeWordScore(word, S) -> int:
```

```
    total = 0
```

```
    multiplier = 1
```

```
    for each placement p in word:
```

```
        base = LETTER_VALUES[p.tile]
```

```
        if isNewPlacement(p, S):
```

```
            bonus = S.getSquare(p.row,p.col).bonusType
```

```
            if bonus == POINTS:
```

```
                base = base * S.getSquare(...).bonusValue
```

```
            if bonus == TIME:
```

```
                TimeManager.addSeconds(bonusValue)
```

```
        total += base
```

```
    total = total * multiplier
```

```
    return total
```

Specific Requirements: 1.3.5.0, 1.3.5.1, 1.3.5.2, 1.3.5.3, 1.3.5.4, 1.3.5.8, 1.3.5.9
Stage Label: Calculation System

GameController.java

```
function processTurn(GameState S, PlacementRequest I):
```

```
    P = buildPlacements(S,I)
    partition = evaluatePredicatesInOrder(S,I,P)
```

```
    if partition != P_valid:
        return S
```

```
    primary = buildPrimaryWord(P,S)
    cross = buildCrossWords(P,S)
    score = computeScore(primary,cross,S)
```

```
    return applyAtomicTransition(S,P,score)
```

```
function applyAtomicTransition(S,P,score):
```

```
    S_copy = deepCopy(S)
```

```
    try:    for p in P:
            S_copy.applyPlacement(p)
```

```
    S_copy.addScore(score)
    evaluateEndCondition(S_copy)
```

```
    return S_copy
```

```
    catch:
        return S    // rollback
```

Specific Requirements: 1.3.1.6, 1.3.1.7, 1.3.3.3, 1.3.4.10, 1.3.4.11, 1.3.8.0, 1.3.8.1,
1.3.8.2, 1.3.8.3, 1.3.8.4, 1.3.8.5, 1.3.1.6, 1.3.1.7, 1.3.8.2

Stage Label: Transition

EndConditionEvaluator.java

```
function evaluateEndCondition(GameState S):
```

```
    if S.getCurrentScore() >= S.getTargetScore():
        S.setGameStatus(COMPLETE)
        S.setGameResult(WIN)
        return
```

```
    if TimeManager.getRemainingSeconds() == 0:
        S.setGameStatus(COMPLETE)
        S.setGameResult(LOSS)
        return
```

Specific Requirements: 1.3.6.4, 1.3.7.0, 1.3.7.1, 1.3.7.2, 1.3.7.3, 1.3.7.4, 1.3.7.5
Stage Label: Persistence Layer

2.3 Decision Tables

1 = condition passes | 0 = condition fails | ? = N/A

Scenario #	Game Running?	Tiles Placed?	In Bounds?	Empty Squares?	Straight Line?	Continuous?	First Move Center Rule Passed?	Connected Rule Passed?	Words Valid?	Result
01	0	?	?	?	?	?	?	?	?	Reject
02	1	0	?	?	?	?	?	?	?	Reject
03	1	1	0	?	?	?	?	?	?	Reject
04	1	1	1	0	?	?	?	?	?	Reject
05	1	1	1	1	0	?	?	?	?	Reject
06	1	1	1	1	1	0	?	?	?	Reject
07	1	1	1	1	1	1	0	?	?	Reject
08	1	1	1	1	1	1	1	0	?	Reject
09	1	1	1	1	1	1	1	1	0	Reject
10	1	1	1	1	1	1	1	1	1	Accept

Decision Table #1: Word Placement Validation

Case	Score $\geq$ Target	Time $\leq$ 0	Result
01	0	0	RUNNING
02	1	0	WON
03	0	1	LOST
04	1	1	WON (priority)

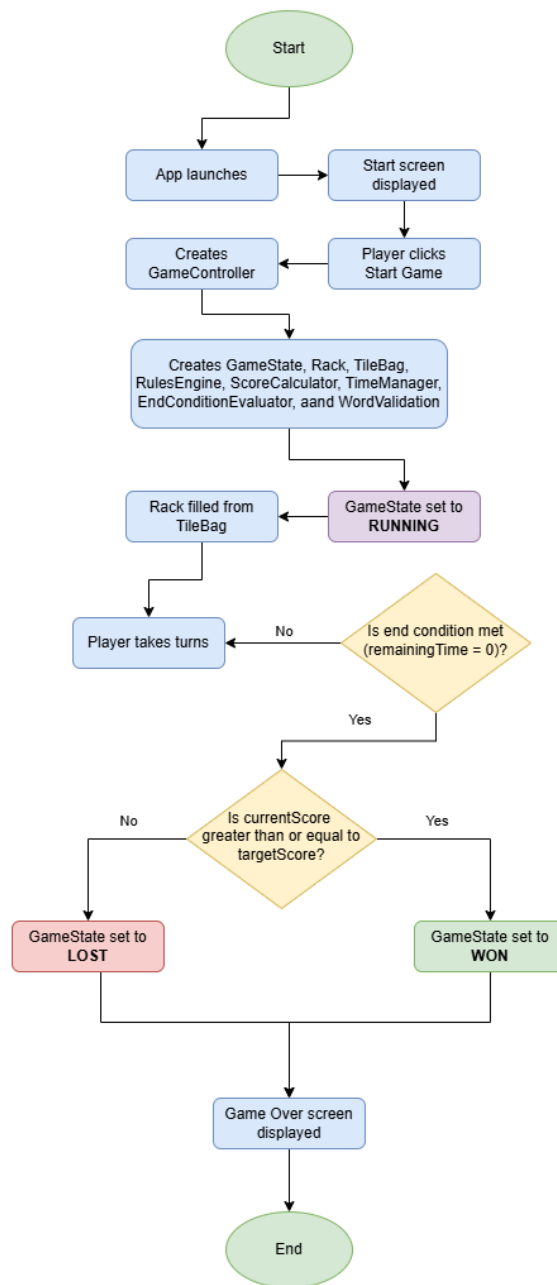
Decision Table #2: Game Outcome

Case	New Tile?	DL	TL	DW	TW	Result
01	0	0	0	0	0	Base score only
02	1	1	0	0	0	Letter $\times 2$
03	1	0	1	0	0	Letter $\times 3$
04	1	0	0	1	0	Word $\times 2$
05	1	0	0	0	1	Word $\times 3$
06	1	1	0	1	0	(Letter $\times 2$) then Word $\times 2$
07	1	0	0	1	1	Word $\times 6$
08	1	?	?	?	?	Bonus consumed after use

Decision Table #3: Scoring

2.4 Control Diagrams

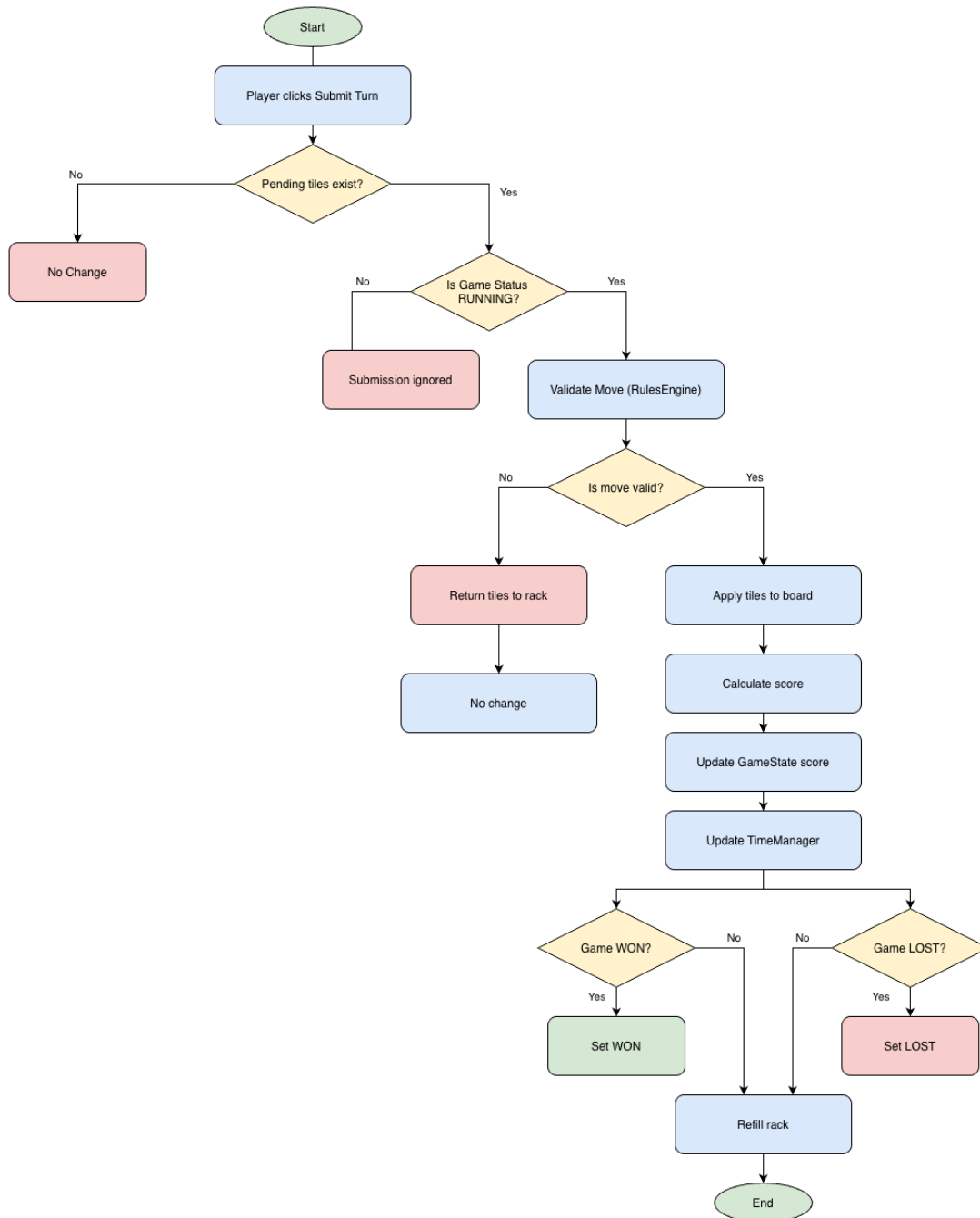
2.4.1 Game State Transition



Requirements:

1.3.1.5, 1.3.6.4, 1.3.7.0-1.3.7.5, 1.3.8.5

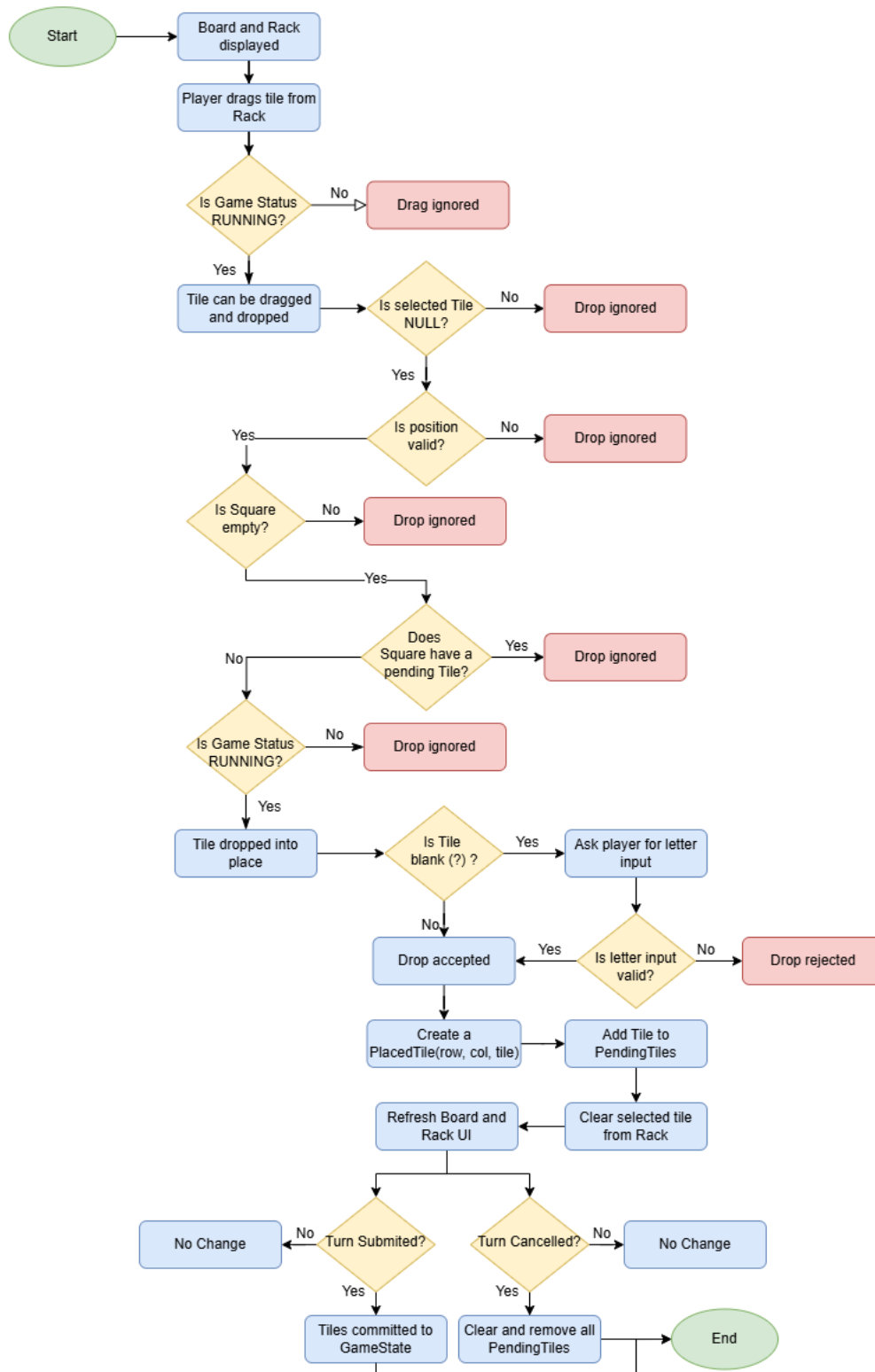
2.4.2 Turn Submission



Requirements:

1.3.4.0, 1.3.4.7, 1.3.5.6, 1.3.6.1, 1.3.6.4, 1.3.7.0-1.3.7.6, 1.3.8.0-1.3.8.5

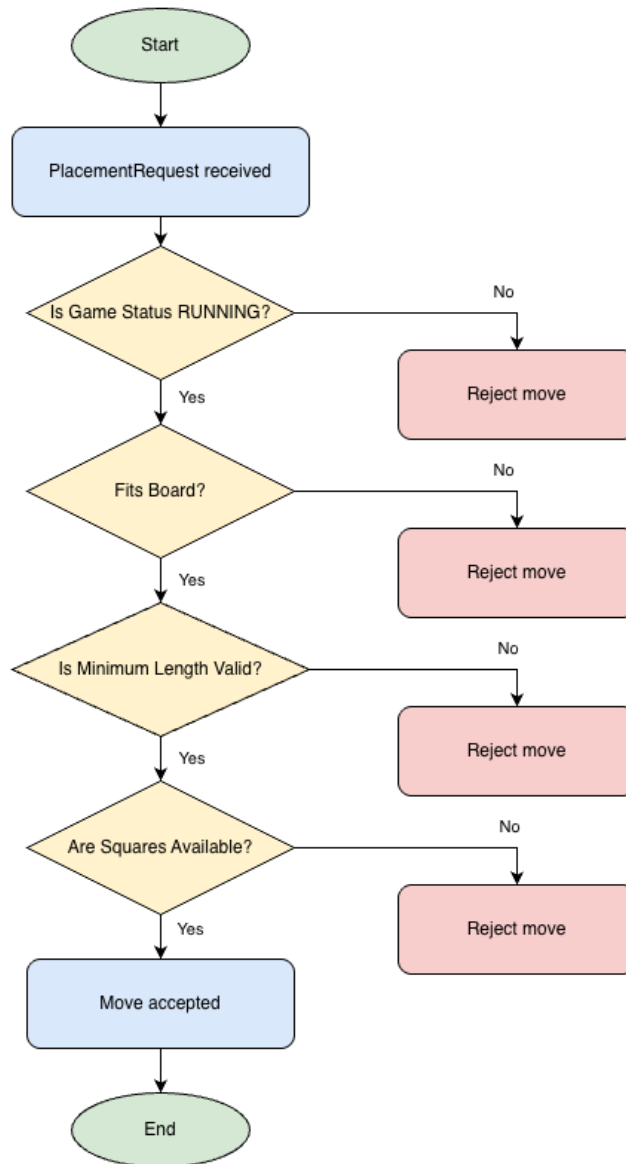
2.4.6 Tile Placement



Requirements:

1.3.9.4-1.3.9.9, 1.3.3.2, 1.3.3.3, 1.3.4.0

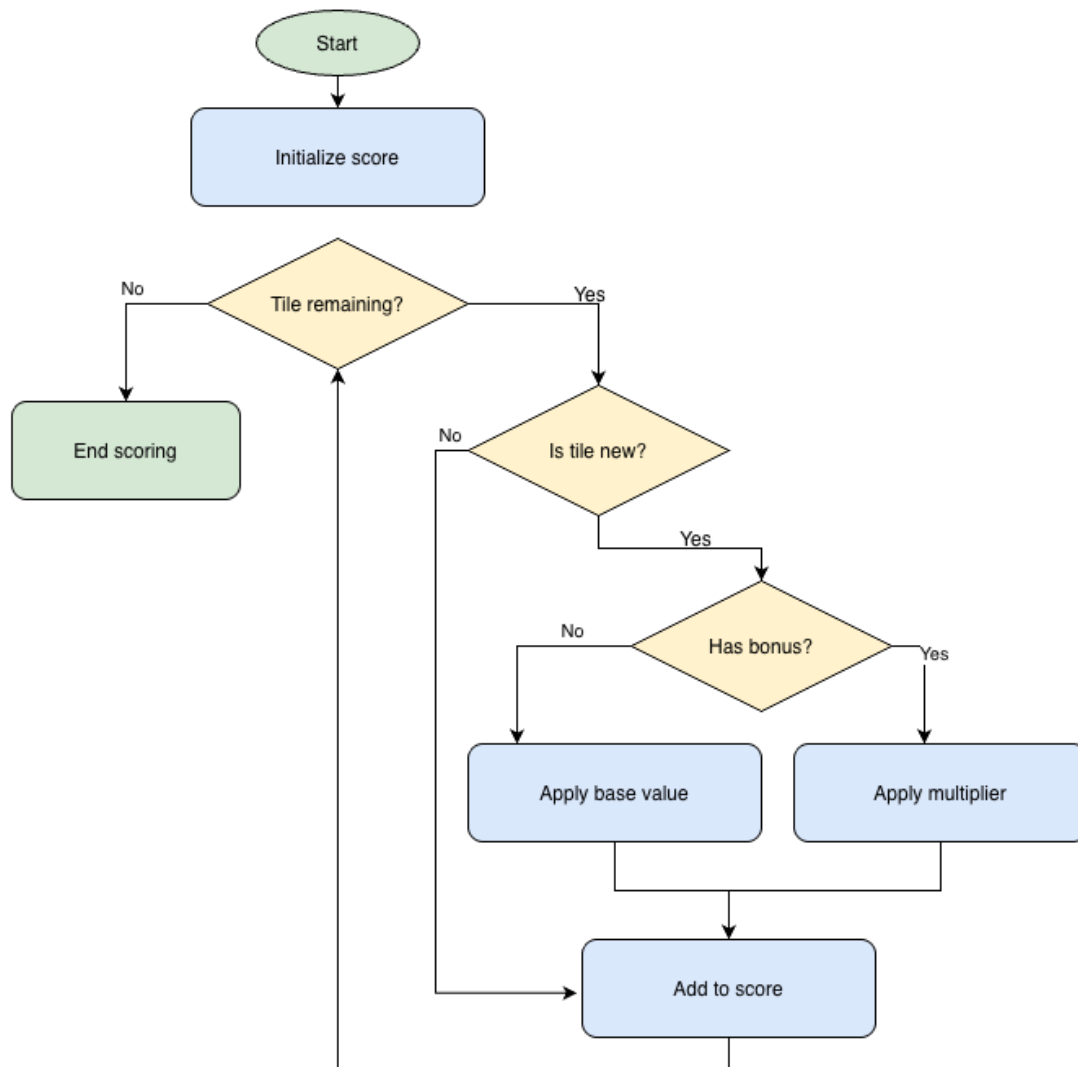
2.4.3 Rules Engine



Requirements:

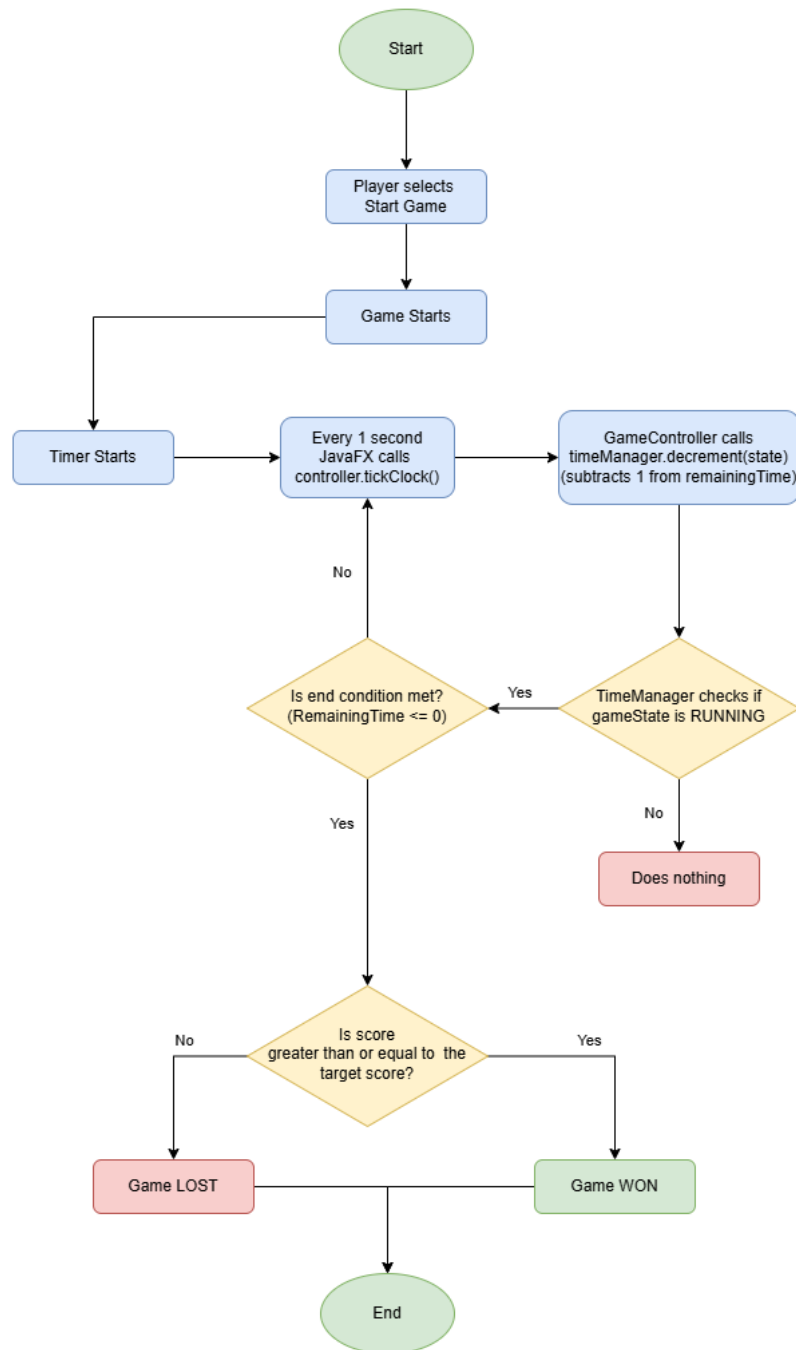
1.3.1.6, 1.3.1.7, 1.3.4.0, 1.3.4.4, 1.3.4.5, 1.3.4.6, 1.3.4.7

2.4.4 Scoring



Requirements:
1.3.5.0-1.3.5.6

2.4.5 Timer



Requirements:

1.3.6.0-1.3.6.4, 1.3.7.0-1.3.7.5

3 Source Code Documentation

GameController.java

```
package csc478.group2;

// 1.3.8 Game Flow Control Requirements
// Class Description: The GameController class manages the overall game by coordinating the
game state, rack, tile bag, rules validation, scoring, time updates, and win/loss
evaluation. It processes submitted turns, along with most game actions.

import java.util.ArrayList;
import java.util.List;

public class GameController {

    private final GameState state;
    private final Rack rack;
    private final TileBag tileBag;
    private final RulesEngine rulesEngine;
    private final ScoreCalculator scoreCalculator;
    private final TimeManager timeManager;
    private final EndConditionEvaluator endEvaluator;
    private final WordValidation wordValidation;

    public GameController() {
        this.state = new GameState();
        this.rack = new Rack();
        this.tileBag = new TileBag();
        this.rulesEngine = new RulesEngine();
        this.scoreCalculator = new ScoreCalculator();
        this.timeManager = new TimeManager();
        this.endEvaluator = new EndConditionEvaluator();
        this.wordValidation = new WordValidation();

        rack.refill(tileBag);
    }

    public GameState getGameState() {
        return state;
    }

    public List<Tile> getRackTiles() {
        return new ArrayList<>(rack.getTiles());
    }

    public void returnTilesToRack(List<PlacedTile> tiles) {
        for (PlacedTile t : tiles) {
            rack.addTile(t.getTile());
        }
    }

    public boolean submitTurn(List<PlacedTile> placedTiles) {

        if (state.getGameStatus() != GameState.GameStatus.RUNNING) {
            return false;
        }
    }
}
```



```

        if (placedTiles == null || placedTiles.isEmpty()) {
            return false;
        }
        // 1.3.8.1 GameController shall call RulesEngine for validation.
        if (!rulesEngine.validateMove(state, placedTiles, wordValidation)) {
            return false;
        }
        // 1.3.8.2 Upon successful validation, GameController shall update GameState.
        applyPlacedTilesToBoard(placedTiles);

        // 1.3.8.3 GameController shall trigger ScoreCalculator for scoring.
        int score = scoreCalculator.calculateTurnScore(state, placedTiles);
        state.setCurrentScore(state.getCurrentScore() + score);

        rack.refill(tileBag);
        // 1.3.8.4 GameController shall trigger TimeManager updates as needed.
        // 1.3.8.5 After each move, GameController shall invoke
EndConditionEvaluator.
        timeManager.tickAfterTurn(state);
        state.setGameStatus(endEvaluator.evaluate(state));

        return true;
    }

    // 1.3.8.0 All placement input shall be processed by GameController.
    private void applyPlacedTilesToBoard(List<PlacedTile> placedTiles) {
        Board board = state.getBoard();

        for (PlacedTile placed : placedTiles) {
            board.placeTile(placed.getRow(), placed.getCol(), placed.getTile());
            rack.removeTile(placed.getTile());
        }
    }

    public void tickClock() {
        timeManager.decrement(state);
        state.setGameStatus(endEvaluator.evaluate(state));
    }
}

```

RulesEngine.java

```
package csc478.group2;

// Class Description: The RulesEngine class is responsible for validating player moves by
// enforcing all game rules, including placement legality, word formation, connectivity, and
// dictionary validation. It ensures that only valid moves are accepted and that no changes are
// made to the game state when a move is invalid. :contentReference[oaicite:0]{index=0}
// Note: Upon combining implementations from each team member, the RulesEngine was modified
// using LLM assistance to better fit specific requirements documentation.

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

public class RulesEngine {

    // 1.3.4.5 The RulesEngine shall validate that the move forms at least one valid
    // word of length two or greater (validateMove).
    // 1.3.4.7 The RulesEngine shall validate that all words formed exist in the
    // dictionary (validateMove).
    // 1.3.4.8 The RulesEngine shall validate all cross-words created by the
    // placement (getAllWordsFormed + validateMove).
    // 1.3.4.10 If validation fails, validateMove shall return false and no state
    // mutation shall occur.
    // 1.3.4.11 If validation succeeds, validateMove shall return true and the
    // GameController shall proceed with scoring and state updates.
    public boolean validateMove(GameState state, List<PlacedTile> placedTiles,
WordValidation dictionary) {
        if (state == null || placedTiles == null || placedTiles.isEmpty() ||
dictionary == null) {
            return false;
        }

        Board board = state.getBoard();
        // 1.3.4.4 The RulesEngine shall validate that all tiles are within board
        // boundaries (validateSquaresAvailable).
        // 1.3.4.6 The RulesEngine shall validate that all placed tiles are on
        // unoccupied squares (validateSquaresAvailable).
        if (!validateSquaresAvailable(board, placedTiles))
            return false;
        if (!validateStraightLine(placedTiles))
            return false;
        if (!validateContinuous(board, placedTiles))
            return false;
        if (!validateFirstMoveCenter(board, placedTiles))
            return false;
        if (!validateConnected(board, placedTiles))
            return false;

        Board tempBoard = copyBoard(board);

        for (PlacedTile placed : placedTiles) {
            tempBoard.placeTile(placed.getRow(), placed.getCol(),
placed.getTile());
        }

        List<String> words = getAllWordsFormed(tempBoard, placedTiles);
    }
}
```

```

        boolean foundWord = false;

        for (String word : words) {
            if (word.length() > 1) {
                foundWord = true;

                if (!dictionary.isValidWord(word)) {
                    return false;
                }
            }
        }

        return foundWord;
    }

    private boolean validateSquaresAvailable(Board board, List<PlacedTile> placedTiles) {
        for (PlacedTile placed : placedTiles) {
            int row = placed.getRow();
            int col = placed.getCol();

            if (!board.isValidPosition(row, col)) {
                return false;
            }

            if (!board.isEmptyAt(row, col)) {
                return false;
            }
        }

        return true;
    }

    private boolean validateStraightLine(List<PlacedTile> placedTiles) {
        boolean sameRow = true;
        boolean sameCol = true;

        int baseRow = placedTiles.get(0).getRow();
        int baseCol = placedTiles.get(0).getCol();

        for (PlacedTile placed : placedTiles) {
            if (placed.getRow() != baseRow)
                sameRow = false;
            if (placed.getCol() != baseCol)
                sameCol = false;
        }

        return sameRow || sameCol;
    }

    private boolean validateContinuous(Board board, List<PlacedTile> placedTiles) {
        Board tempBoard = copyBoard(board);

        for (PlacedTile placed : placedTiles) {
            tempBoard.placeTile(placed.getRow(), placed.getCol(),
placed.getTile());
        }

        boolean sameRow = true;
        boolean sameCol = true;

        int baseRow = placedTiles.get(0).getRow();

```

```

        int baseCol = placedTiles.get(0).getCol();

        for (PlacedTile placed : placedTiles) {
            if (placed.getRow() != baseRow)
                sameRow = false;
            if (placed.getCol() != baseCol)
                sameCol = false;
        }

        if (!sameRow && !sameCol) {
            return false;
        }

        if (sameRow) {
            int minCol = Integer.MAX_VALUE;
            int maxCol = Integer.MIN_VALUE;

            for (PlacedTile placed : placedTiles) {
                minCol = Math.min(minCol, placed.getCol());
                maxCol = Math.max(maxCol, placed.getCol());
            }

            for (int col = minCol; col <= maxCol; col++) {
                if (tempBoard.isEmptyAt(baseRow, col)) {
                    return false;
                }
            }
        }
        else {
            int minRow = Integer.MAX_VALUE;
            int maxRow = Integer.MIN_VALUE;

            for (PlacedTile placed : placedTiles) {
                minRow = Math.min(minRow, placed.getRow());
                maxRow = Math.max(maxRow, placed.getRow());
            }

            for (int row = minRow; row <= maxRow; row++) {
                if (tempBoard.isEmptyAt(row, baseCol)) {
                    return false;
                }
            }
        }
    }

    return true;
}

private boolean validateFirstMoveCenter(Board board, List<PlacedTile> placedTiles) {
    if (!board.isEmpty()) {
        return true;
    }

    for (PlacedTile placed : placedTiles) {
        if (placed.getRow() == 7 && placed.getCol() == 7) {
            return true;
        }
    }

    return false;
}

```

```

private boolean validateConnected(Board board, List<PlacedTile> placedTiles) {
    if (board.isEmpty()) {
        return true;
    }

    for (PlacedTile placed : placedTiles) {
        int row = placed.getRow();
        int col = placed.getCol();

        if ((row > 0 && !board.isEmptyAt(row - 1, col)) || (row < Board.SIZE -
1 && !board.isEmptyAt(row + 1, col))
            || (col > 0 && !board.isEmptyAt(row, col - 1))
            || (col < Board.SIZE - 1 && !board.isEmptyAt(row, col +
1))) {
                return true;
            }
    }

    return false;
}

// 1.3.4.8 The RulesEngine shall validate all cross-words created by the
// placement (getAllWordsFormed + validateMove).
// 1.3.4.9 The RulesEngine shall construct all words formed from the placement
// for validation (getAllWordsFormed).
private List<String> getAllWordsFormed(Board board, List<PlacedTile> placedTiles) {
    Set<String> words = new HashSet<>();

    boolean sameRow = true;
    boolean sameCol = true;

    int baseRow = placedTiles.get(0).getRow();
    int baseCol = placedTiles.get(0).getCol();

    for (PlacedTile placed : placedTiles) {
        if (placed.getRow() != baseRow)
            sameRow = false;
        if (placed.getCol() != baseCol)
            sameCol = false;
    }

    PlacedTile first = placedTiles.get(0);

    if (sameRow) {
        String mainWord = buildWord(board, first.getRow(), first.getCol(), 0,
1);

        if (mainWord.length() > 1) {
            words.add(mainWord);
        }

        for (PlacedTile placed : placedTiles) {
            String crossWord = buildWord(board, placed.getRow(),
placed.getCol(), 1, 0);

            if (crossWord.length() > 1) {
                words.add(crossWord);
            }
        }
    } else if (sameCol) {

```

```

        String mainWord = buildWord(board, first.getRow(), first.getCol(), 1,
0);

        if (mainWord.length() > 1) {
            words.add(mainWord);
        }

        for (PlacedTile placed : placedTiles) {
            String crossWord = buildWord(board, placed.getRow(),
placed.getCol(), 0, 1);

            if (crossWord.length() > 1) {
                words.add(crossWord);
            }
        }

        return new ArrayList<>(words);
    }

    private String buildWord(Board board, int row, int col, int dRow, int dCol) {
        int currentRow = row;
        int currentCol = col;

        while (board.isValidPosition(currentRow - dRow, currentCol - dCol)
            && !board.isEmptyAt(currentRow - dRow, currentCol - dCol)) {
            currentRow -= dRow;
            currentCol -= dCol;
        }

        StringBuilder word = new StringBuilder();

        while (board.isValidPosition(currentRow, currentCol) &&
!board.isEmptyAt(currentRow, currentCol)) {
            word.append(board.getCell(currentRow, currentCol));
            currentRow += dRow;
            currentCol += dCol;
        }

        return word.toString();
    }

    private Board copyBoard(Board board) {
        Board copy = new Board();

        for (int row = 0; row < Board.SIZE; row++) {
            for (int col = 0; col < Board.SIZE; col++) {
                Square original = board.getSquare(row, col);

                if (original.isOccupied()) {
                    copy.placeTile(row, col, original.getTile());
                }
            }
        }

        return copy;
    }
}

```

ScoreCalculator.java

```

package csc478.group2;

// 1.3.5 Scoring Requirements
// The ScoreCalculator class is responsible for determining the total score of a turn by
// evaluating all words formed from a placement.

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

// 1.3.5.0 All scoring shall be handled by ScoreCalculator.
// 1.3.5.9 Scoring logic shall be deterministic and independent of the UI layer.
public class ScoreCalculator {
    // 1.3.5.1 The ScoreCalculator shall calculate the total score for a submitted
    // turn using calculateTurnScore(GameState, List<PlacedTile>).
    // 1.3.5.3 The ScoreCalculator shall compute each word score by summing the
    // point values of all tiles in that word.
    public int calculateTurnScore(GameState state, List<PlacedTile> placedTiles) {
        Board board = state.getBoard();

        List<WordData> words = getWordsFormed(board, placedTiles);

        int total = 0;

        for (WordData word : words) {
            int wordScore = 0;
            int wordMultiplier = 1;

            // 1.3.5.4 The ScoreCalculator shall apply letter bonuses and word
            // to newly placed tiles on unused bonus squares.
            // 1.3.5.5 The ScoreCalculator shall prevent duplicate word scoring by
            // already-scored word positions.
            for (Cell cell : word.cells) {
                Square square = board.getSquare(cell.row, cell.col);
                Tile tile = square.getTile();

                int letterScore = tile.getPointValue();

                if (isNewTile(placedTiles, cell.row, cell.col) &&
                    !square.isBonusUsed()) {
                    switch (square.getBonusType()) {
                        case DOUBLE_LETTER -> letterScore *= 2;
                        case TRIPLE_LETTER -> letterScore *= 3;
                        case DOUBLE_WORD -> wordMultiplier *= 2;
                        case TRIPLE_WORD -> wordMultiplier *= 3;
                        default -> {
                        }
                    }
                }

                wordScore += letterScore;
            }

            total += wordScore * wordMultiplier;
        }
    }
}

```

```

// 1.3.5.6 The ScoreCalculator shall add a 50-point bonus when all seven rack
// tiles are used in one turn.
if (placedTiles.size() == 7) {
    total += 50;
}
// 1.3.5.7 The ScoreCalculator shall consume used bonus squares after scoring
so
// bonuses apply only once.
for (PlacedTile placed : placedTiles) {
    Square square = board.getSquare(placed.getRow(), placed.getCol());
    if (!square.isBonusUsed()) {
        square.consumeBonus();
    }
}
// 1.3.5.8 The ScoreCalculator shall return the total score for the move.
return total;
}

// 1.3.5.2 The ScoreCalculator shall identify all words formed by the move using
// getWordsFormed(Board, List<PlacedTile>).
private List<WordData> getWordsFormed(Board board, List<PlacedTile> placedTiles) {
    List<WordData> words = new ArrayList<>();
    Set<String> seen = new HashSet<>();

    boolean sameRow = true;
    boolean sameCol = true;

    int baseRow = placedTiles.get(0).getRow();
    int baseCol = placedTiles.get(0).getCol();

    for (PlacedTile placed : placedTiles) {
        if (placed.getRow() != baseRow)
            sameRow = false;
        if (placed.getCol() != baseCol)
            sameCol = false;
    }

    PlacedTile first = placedTiles.get(0);

    if (sameRow) {
        WordData mainWord = buildWord(board, first.getRow(), first.getCol(),
0, 1);
        addWordIfValid(words, seen, mainWord);

        for (PlacedTile placed : placedTiles) {
            WordData crossWord = buildWord(board, placed.getRow(),
placed.getCol(), 1, 0);
            addWordIfValid(words, seen, crossWord);
        }

    } else if (sameCol) {
        WordData mainWord = buildWord(board, first.getRow(), first.getCol(),
1, 0);
        addWordIfValid(words, seen, mainWord);

        for (PlacedTile placed : placedTiles) {
            WordData crossWord = buildWord(board, placed.getRow(),
placed.getCol(), 0, 1);
            addWordIfValid(words, seen, crossWord);
        }
    }
}

```



```

        return words;
    }

    private void addWordIfValid(List<WordData> words, Set<String> seen, WordData word) {
        if (word == null || word.word.length() < 2) {
            return;
        }

        String key = word.getKey();

        if (!seen.contains(key)) {
            seen.add(key);
            words.add(word);
        }
    }

    private WordData buildWord(Board board, int row, int col, int dRow, int dCol) {
        int currentRow = row;
        int currentCol = col;

        while (board.isValidPosition(currentRow - dRow, currentCol - dCol)
            && !board.isEmptyAt(currentRow - dRow, currentCol - dCol)) {
            currentRow -= dRow;
            currentCol -= dCol;
        }

        StringBuilder word = new StringBuilder();
        List<Cell> cells = new ArrayList<>();

        while (board.isValidPosition(currentRow, currentCol) &&
            !board.isEmptyAt(currentRow, currentCol)) {

            word.append(board.getCell(currentRow, currentCol));
            cells.add(new Cell(currentRow, currentCol));

            currentRow += dRow;
            currentCol += dCol;
        }

        return new WordData(word.toString(), cells);
    }

    private boolean isNewTile(List<PlacedTile> placedTiles, int row, int col) {
        for (PlacedTile placed : placedTiles) {
            if (placed.getRow() == row && placed.getCol() == col) {
                return true;
            }
        }
        return false;
    }

    private static class WordData {
        private final String word;
        private final List<Cell> cells;

        private WordData(String word, List<Cell> cells) {
            this.word = word;
            this.cells = cells;
        }
    }

```

```
        private String getKey() {
            StringBuilder key = new StringBuilder();

            for (Cell cell : cells) {
                key.append(cell.row).append(",").append(cell.col).append(";");
            }

            return key.toString();
        }

        private static class Cell {
            private final int row;
            private final int col;

            private Cell(int row, int col) {
                this.row = row;
                this.col = col;
            }
        }
    }
```

EndConditionEvaluator.java

```
package csc478.group2;

// 1.3.7 End Condition Requirements
// Class Description: The EndConditionEvaluator class determines the current game outcome by
// evaluating whether the player has met the win condition or lose condition, and returns the
// appropriate game status.

public class EndConditionEvaluator {

    // 1.3.7.0 EndConditionEvaluator shall check win condition using
    // checkWinCondition(score, targetScore).
    public boolean checkWinCondition(int score, int targetScore) {
        // 1.3.7.1 A win shall occur when currentScore >= targetScore.
        return score >= targetScore;
    }

    // 1.3.6.4 If time expires, the EndConditionEvaluator shall update the GameState
    // to LOST.
    // 1.3.7.2 EndConditionEvaluator shall check lose condition using
    // checkLoseCondition(timeRemaining).
    public boolean checkLoseCondition(int timeRemaining) {
        // 1.3.7.3 A loss shall occur when remainingTime <= 0.
        return timeRemaining <= 0;
    }

    // 1.3.7.4 Upon win, GameState.gameStatus shall be set to WON.
    // 1.3.7.5 Upon loss, GameState.gameStatus shall be set to LOST.
    public GameState.GameStatus evaluate(GameState state) {
        if (checkWinCondition(state.getCurrentScore(), state.getTargetScore())) {
            return GameState.GameStatus.WON;
        }
        if (checkLoseCondition(state.getRemainingTime())) {
            return GameState.GameStatus.LOST;
        }
        return GameState.GameStatus.RUNNING;
    }
}
```

TimeManager.java

```
package csc478.group2;

// 1.3.6 Time System Requirements
// Class Description: The TimeManager class is responsible for managing the game's timing by
// decrementing remaining time and determining when time has expired.
// 1.3.6.0 The GameState shall initialize remainingTime to the initial time limit at game
// start.
public class TimeManager {
    // 1.3.6.1 The TimeManager shall decrement remaining time only when the game
    // status is RUNNING (decrement).
    public void decrement(GameState state) {
        if (state.getGameStatus() != GameState.GameStatus.RUNNING) {
            return;
        }

        state.setRemainingTime(state.getRemainingTime() - 1);
    }

    // 1.3.6.2 The TimeManager shall update time after each turn using
    // tickAfterTurn(GameState).
    public void tickAfterTurn(GameState state) {
        decrement(state);
    }

    // 1.3.6.3 The TimeManager shall determine if time has expired using
    // checkTimeExpired(GameState).
    public boolean checkTimeExpired(GameState state) {
        return state.getRemainingTime() <= 0;
    }
}
```

GameState.java

```
package csc478.group2;

// 1.3.1 Core Game State Requirements
// Class Description: The GameState class serves as the central source of truth for the
// game, storing the current score, remaining time, target score, board state, and overall game
// status. It is updated only by core logic components to ensure consistent and controlled game
// behavior.
// 1.3.1.6 Only the RulesEngine and GameController shall modify GameState.
// 1.3.1.7 Invalid placement attempts shall not modify GameState.

public class GameState {

    public enum GameStatus {
        RUNNING, WON, LOST
    }

    private int currentScore;
    private int remainingTime;
    private int targetScore;
    private final Board board;
    private GameStatus gameStatus;

    // 1.3.1.0 The system shall maintain a single GameState object as the
    // authoritative source of truth for all game data.
    public GameState() {
        this.board = new Board();
        this.currentScore = 0;
        this.remainingTime = 300;
        this.targetScore = 150;
        this.gameStatus = GameStatus.RUNNING;
    }

    // 1.3.1.1 The GameState shall contain currentScore.
    public int getCurrentScore() {
        return currentScore;
    }

    // 1.3.1.2 The GameState shall contain remainingTime.
    public int getRemainingTime() {
        return remainingTime;
    }

    // 1.3.1.3 The GameState shall contain targetScore.
    public int getTargetScore() {
        return targetScore;
    }

    // 1.3.1.4 The GameState shall contain boardState.
    public Board getBoard() {
        return board;
    }

    // 1.3.1.5 The GameState shall contain gameStatus with possible values: RUNNING,
    // WON, LOST.
    public GameStatus getGameStatus() {
        return gameStatus;
    }
}
```

```
// 1.3.1.1 The GameState shall contain currentScore.
void setCurrentScore(int currentScore) {
    this.currentScore = currentScore;
}

// 1.3.1.2 The GameState shall contain remainingTime.
void setRemainingTime(int remainingTime) {
    this.remainingTime = remainingTime;
}

// 1.3.1.5 The GameState shall contain gameStatus with possible values: RUNNING,
// WON, LOST.
void setGameStatus(GameStatus gameStatus) {
    this.gameStatus = gameStatus;
}
}
```

Board.java

```

package csc478.group2;

// 1.3.2 Board and Square Requirements
// Class Description: The primary purpose of the Board class is to represent the game grid
// and manage all squares, including tile placement, bonus types, and check whether tiles are
// empty.

public class Board {

    public static final int SIZE = 15;
    public static final char EMPTY_CELL = '\0';

    private final Square[][] grid;

    // 1.3.2.0 The system shall initialize a 15x15 2D Board grid at game start.
    public Board() {
        // 1.3.2.1 The Board shall contain Square objects.
        // 1.3.2.2 Each Square shall maintain row and column position.
        grid = new Square[SIZE][SIZE];

        for (int r = 0; r < SIZE; r++) {
            for (int c = 0; c < SIZE; c++) {
                grid[r][c] = new Square(r, c, Square.BonusType.NONE, 0);
            }
        }

        initializeBonuses();
    }

    // 1.3.2.4 Each Square shall maintain a bonusValue.
    private void initializeBonuses() {
        int[][] tw = { { 0, 0 }, { 0, 7 }, { 0, 14 }, { 7, 0 }, { 7, 14 }, { 14, 0 },
            { 14, 7 }, { 14, 14 } };

        for (int[] p : tw) {
            setBonus(p[0], p[1], Square.BonusType.TRIPLE_WORD, 3);
        }

        int[][] dw = { { 1, 1 }, { 2, 2 }, { 3, 3 }, { 4, 4 }, { 10, 10 }, { 11, 11 },
            { 12, 12 }, { 13, 13 }, { 1, 13 }, { 2, 12 }, { 3, 11 }, { 4, 10 }, { 10, 4 }, { 11, 3 },
            { 12, 2 }, { 13, 1 }, { 7, 7 } };
        for (int[] p : dw) {
            setBonus(p[0], p[1], Square.BonusType.DOUBLE_WORD, 2);
        }

        int[][] tl = { { 1, 5 }, { 1, 9 }, { 5, 1 }, { 5, 5 }, { 5, 9 }, { 5, 13 }, {
            9, 1 }, { 9, 5 }, { 9, 9 }, { 9, 13 }, { 13, 5 }, { 13, 9 } };
        for (int[] p : tl) {
            setBonus(p[0], p[1], Square.BonusType.TRIPLE_LETTER, 3);
        }

        int[][] dl = { { 0, 3 }, { 0, 11 }, { 2, 6 }, { 2, 8 }, { 3, 0 }, { 3, 7 }, {
            3, 14 }, { 6, 2 }, { 6, 6 }, { 6, 8 }, { 6, 12 }, { 7, 3 }, { 7, 11 }, { 8, 2 }, { 8, 6 }, {
            8, 8 }, { 8, 12 }, { 11, 0 }, { 11, 7 }, { 11, 14 }, { 12, 6 }, { 12, 8 }, { 14, 3 }, { 14, 11 } };
    }
}

```

```

        for (int[] p : dl) {
            setBonus(p[0], p[1], Square.BonusType.DOUBLE_LETTER, 2);
        }

    }

    // 1.3.2.6 Bonus squares shall apply their effect only once when first used.
    private void setBonus(int row, int col, Square.BonusType type, int value) {
        grid[row][col] = new Square(row, col, type, value);
    }

    public Square getSquare(int r, int c) {
        return grid[r][c];
    }

    // 1.3.2.3 Each Square shall maintain an occupied flag.
    // 1.3.2.5 A Square marked as occupied shall not accept additional tiles.
    // 1.3.3.2 Tiles shall only be placed onto unoccupied Squares.
    public boolean isValidPosition(int r, int c) {
        return r >= 0 && r < SIZE && c >= 0 && c < SIZE;
    }

    // 1.3.2.3 Each Square shall maintain an occupied flag.
    // 1.3.2.5 A Square marked as occupied shall not accept additional tiles.
    public boolean isEmptyAt(int r, int c) {
        return !grid[r][c].isOccupied();
    }

    public boolean placeTile(int r, int c, Tile tile) {
        return grid[r][c].placeTile(tile);
    }

    public void clearPosition(int r, int c) {
        grid[r][c].removeTile();
    }

    public char getCell(int r, int c) {
        return grid[r][c].isOccupied() ? grid[r][c].getLetter() : EMPTY_CELL;
    }

    // 1.3.2.3 Each Square shall maintain an occupied flag.
    // 1.3.2.5 A Square marked as occupied shall not accept additional tiles.
    public boolean isEmpty() {
        for (Square[] row : grid) {
            for (Square square : row) {
                if (square.isOccupied()) {
                    return false;
                }
            }
        }
        return true;
    }
}

```


Square.java

```
package csc478.group2;

// Class Description: The Square class represents a single position on the board, storing
// its location, whether it is occupied, the tile placed on it, and any bonus type or value
// that may affect scoring.

public class Square {

    public enum BonusType {
        NONE, DOUBLE_LETTER, TRIPLE_LETTER, DOUBLE_WORD, TRIPLE_WORD, TIME
    }

    private final int row;
    private final int col;
    private boolean occupied;
    private Tile tile;
    private BonusType bonusType;
    private int bonusValue;
    private boolean bonusUsed;

    public Square(int row, int col, BonusType bonusType, int bonusValue) {
        this.row = row;
        this.col = col;
        this.bonusType = bonusType;
        this.bonusValue = bonusValue;
        this.occupied = false;
        this.tile = null;
        this.bonusUsed = false;
    }

    public int getRow() {
        return row;
    }

    public int getCol() {
        return col;
    }

    public boolean isOccupied() {
        return occupied;
    }

    public Tile getTile() {
        return tile;
    }

    public boolean placeTile(Tile t) {
        if (occupied)
            return false;
        tile = t;
        occupied = true;
        return true;
    }

    public Tile removeTile() {
        Tile old = tile;
        tile = null;
        occupied = false;
    }
}
```

```
        return old;
    }

    public char getLetter() {
        return tile == null ? Board.EMPTY_CELL : tile.getLetter();
    }

    public BonusType getBonusType() {
        return bonusType;
    }

    public int getBonusValue() {
        return bonusValue;
    }

    public boolean isBonusUsed() {
        return bonusUsed;
    }

    public void consumeBonus() {
        bonusUsed = true;
        bonusType = BonusType.NONE;
        bonusValue = 0;
    }
}
```

Tile.java

```
package csc478.group2;

//1.3.3
//Class Description: The primary purpose of the Tile class is to represent an individual
letter tile, storing its character and point value. It is used for placing letters on the
board and calculating scores.

public class Tile {
    // 1.3.3.0 Each Tile shall contain a character.
    private final char letter;
    // 1.3.3.1 Each Tile shall derive scoring from word length logic.
    private final int pointValue;
    private final boolean blank;

    public Tile(char letter, int pointValue) {
        this(letter, pointValue, false);
    }

    public Tile(char letter, int pointValue, boolean blank) {
        this.letter = Character.toUpperCase(letter);
        this.pointValue = pointValue;
        this.blank = blank;
    }

    public char getLetter() {
        return letter;
    }

    public int getPointValue() {
        return blank ? 0 : pointValue;
    }

    public boolean isBlank() {
        return blank;
    }

    public Tile asBlankLetter(char chosenLetter) {
        if (!blank) {
            return this;
        }

        return new Tile(Character.toUpperCase(chosenLetter), 0, true);
    }
}
```

Rack.java

```
package csc478.group2;

// Class Description: The Rack class manages the current collection of tiles, allowing tiles
// to be added, removed, refilled from the tile bag, and accessed during gameplay.

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Rack {

    private static final int MAX_TILES = 7;

    private final List<Tile> tiles;

    public Rack() {
        this.tiles = new ArrayList<>();
    }

    public List<Tile> getTiles() {
        return tiles;
    }

    public boolean isFull() {
        return tiles.size() >= MAX_TILES;
    }

    public boolean isEmpty() {
        return tiles.isEmpty();
    }

    public int size() {
        return tiles.size();
    }

    public void refill(TileBag tileBag) {
        while (tiles.size() < MAX_TILES && !tileBag.isEmpty()) {
            Tile drawn = tileBag.drawTile();
            if (drawn != null) {
                tiles.add(drawn);
            }
        }
    }

    public void addTile(Tile tile) {
        if (tile != null && tiles.size() < MAX_TILES) {
            tiles.add(tile);
        }
    }

    public boolean removeTile(Tile tile) {
        return tiles.remove(tile);
    }

    public Tile getTileAt(int index) {
        if (index < 0 || index >= tiles.size()) {
            return null;
        }
    }
}
```

```
        return tiles.get(index);
    }

    public void clear() {
        tiles.clear();
    }

    public List<Tile> snapshot() {
        return new ArrayList<>(tiles);
    }

    public void restore(List<Tile> savedTiles) {
        tiles.clear();
        tiles.addAll(savedTiles);
    }

    public void sortAlphabetically() {
        Collections.sort(tiles, (a, b) -> Character.compare(a.getLetter(),
b.getLetter()));
    }
}
```

TileBag.java

```
package csc478.group2;

// Class Description: The TileBag class manages the collection of all tiles in the game,
// allowing tiles to be randomly drawn, returned, and initialized according to standard
// Scrabble distribution.

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class TileBag {

    private final List<Tile> tiles;

    public TileBag() {
        this.tiles = createStandardBag();
        Collections.shuffle(this.tiles);
    }

    public boolean isEmpty() {
        return tiles.isEmpty();
    }

    public int size() {
        return tiles.size();
    }

    public Tile drawTile() {
        if (tiles.isEmpty()) {
            return null;
        }
        return tiles.remove(0);
    }

    public void returnTile(Tile tile) {
        if (tile != null) {
            tiles.add(tile);
        }
    }

    public void returnTiles(List<Tile> returnedTiles) {
        if (returnedTiles == null) {
            return;
        }

        for (Tile tile : returnedTiles) {
            if (tile != null) {
                tiles.add(tile);
            }
        }
    }

    private List<Tile> createStandardBag() {
        List<Tile> bag = new ArrayList<>();

        addTiles(bag, 'A', 1, 9);
        addTiles(bag, 'B', 3, 2);
        addTiles(bag, 'C', 3, 2);
    }
}
```

```
        addTiles(bag, 'D', 2, 4);
        addTiles(bag, 'E', 1, 12);
        addTiles(bag, 'F', 4, 2);
        addTiles(bag, 'G', 2, 3);
        addTiles(bag, 'H', 4, 2);
        addTiles(bag, 'I', 1, 9);
        addTiles(bag, 'J', 8, 1);
        addTiles(bag, 'K', 5, 1);
        addTiles(bag, 'L', 1, 4);
        addTiles(bag, 'M', 3, 2);
        addTiles(bag, 'N', 1, 6);
        addTiles(bag, 'O', 1, 8);
        addTiles(bag, 'P', 3, 2);
        addTiles(bag, 'Q', 10, 1);
        addTiles(bag, 'R', 1, 6);
        addTiles(bag, 'S', 1, 4);
        addTiles(bag, 'T', 1, 6);
        addTiles(bag, 'U', 1, 4);
        addTiles(bag, 'V', 4, 2);
        addTiles(bag, 'W', 4, 2);
        addTiles(bag, 'X', 8, 1);
        addTiles(bag, 'Y', 4, 2);
        addTiles(bag, 'Z', 10, 1);

        bag.add(new Tile('?', 0, true));
        bag.add(new Tile('?', 0, true));

        return bag;
    }

    private void addTiles(List<Tile> bag, char letter, int value, int count) {
        for (int i = 0; i < count; i++) {
            bag.add(new Tile(letter, value));
        }
    }
}
```

PlacementRequest.java

```
package csc478.group2;

// 1.3.4 Word Placement and Validation Requirements
// Class Description: Processes placement requests.
// 1.3.3.3 Tile placement shall occur only through validated PlacementRequest processing.
// 1.3.4.0 All moves shall be submitted through a PlacementRequest.
public class PlacementRequest {
    // 1.3.4.1 A PlacementRequest shall include a word string.
    public String word;
    // 1.3.4.2 A PlacementRequest shall include a starting position.
    public int row;
    public int col;
    // 1.3.4.3 A PlacementRequest shall include a direction (horizontal or
    // vertical).
    public boolean horizontal;
}
```

PlacedTile.java

```
package csc478.group2;

// Class Description: The PlacedTile class represents a tile placed on the board during a
// turn, storing its row, column position, and associated tile.

public class PlacedTile {
    private final int row;
    private final int col;
    private final Tile tile;

    public PlacedTile(int row, int col, Tile tile) {
        this.row = row;
        this.col = col;
        this.tile = tile;
    }

    public int getRow() {
        return row;
    }

    public int getCol() {
        return col;
    }

    public Tile getTile() {
        return tile;
    }
}
```


WordValidation.java

```
package csc478.group2;

// Class Description: The WordValidation class loads a dictionary of valid words and
// verifies whether a given word is valid during gameplay.

import java.io.BufferedReader;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.util.HashSet;
import java.util.Set;

public class WordValidation {

    private final Set<String> validWords = new HashSet<>();

    public WordValidation() {
        loadWords();
    }

    private void loadWords() {
        try (InputStream input =
getClass().getResourceAsStream("/csc478/group2/sowpods.txt")) {
            if (input == null) {
                throw new IllegalStateException("Could not find sowpods.txt");
            }

            try (BufferedReader reader = new BufferedReader(new
InputStreamReader(input))) {
                String line;
                while ((line = reader.readLine()) != null) {
                    validWords.add(line.trim().toUpperCase());
                }
            }
        } catch (Exception e) {
            throw new RuntimeException("Failed to load SOWPODS", e);
        }
    }

    public boolean isValidWord(String word) {
        return validWords.contains(word.toUpperCase());
    }
}
```

JavaFXMain.java

```
package csc478.group2;

// 1.3.9 UI Layer Requirements
// Class Description: Handles all user interaction and rendering for the Scrabble game.
// This class displays game data and collects input through drag-and-drop, but does not
// directly commit changes to GameState.
// Note: The process for creating this class involved significant leveraging of LLMs for
// formatting assistance.

import javafx.animation.KeyFrame;
import javafx.animation.Timeline;
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.TextInputDialog;
import javafx.scene.input.ClipboardContent;
import javafx.scene.input.Dragboard;
import javafx.scene.input.TransferMode;
import javafx.scene.layout.GridPane;
import javafx.scene.layout.HBox;
import javafx.scene.layout.StackPane;
import javafx.scene.layout.VBox;
import javafx.scene.text.Font;
import javafx.stage.Stage;
import javafx.util.Duration;
import java.util.ArrayList;
import java.util.List;

public class JavaFX extends Application {

    private static final int CELL_SIZE = 40;

    private GameController controller;
    private Stage primaryStage;

    private GridPane boardGrid;
    private StackPane[][] boardCells;
    private HBox rackBox;

    private Label scoreLabel;
    private Label timerLabel;
    private Label messageLabel;

    private Timeline timeline;

    // Stores tiles placed during the current turn before they are submitted.
    private final List<PlacedTile> pendingTiles = new ArrayList<>();
    private Tile selectedTileUI;

    // Starts the JavaFX application and displays the start screen.
    @Override
    public void start(Stage primaryStage) {
        this.primaryStage = primaryStage;
        showStartScreen();
    }
}
```

```

// Displays the start menu before the game begins.
private void showStartScreen() {
    VBox root = new VBox(20);
    root.setAlignment(Pos.CENTER);
    root.setPadding(new Insets(30));

    Label title = new Label("Scrabble");
    title.setFont(Font.font(36));

    Button startButton = new Button("Start Game");
    startButton.setFont(Font.font(18));

    startButton.setOnAction(e -> startGame());

    root.getChildren().addAll(title, startButton);

    primaryStage.setScene(new Scene(root, 950, 800));
    primaryStage.setTitle("Scrabble");
    primaryStage.show();
}

// Initializes a new game and builds the main gameplay screen.
private void startGame() {
    controller = new GameController();
    pendingTiles.clear();
    selectedTileUI = null;

    VBox root = new VBox(10);
    root.setPadding(new Insets(10));
    root.setAlignment(Pos.CENTER);

    // Header displays the title, score, and timer.
    VBox header = new VBox(5);
    header.setAlignment(Pos.CENTER);
    header.setPadding(new Insets(10));
    header.setStyle("-fx-background-color: linear-gradient(to right, #4b0082,
#8a2be2);");

    Label titleLabel = new Label("SCRABBLE");
    titleLabel.setFont(Font.font("Arial", 32));
    titleLabel.setStyle("-fx-text-fill: white; -fx-font-weight: bold;");

    HBox infoRow = new HBox(40);
    infoRow.setAlignment(Pos.CENTER);

    // 1.3.9.1 The JavaFX UI shall display currentScore using scoreLabel in
    // refreshUI.
    scoreLabel = new Label();
    scoreLabel.setFont(Font.font("Arial", 18));
    scoreLabel.setStyle("-fx-text-fill: white;");

    // 1.3.9.2 The JavaFX UI shall display remainingTime using timerLabel in
    // refreshUI.
    timerLabel = new Label();
    timerLabel.setFont(Font.font("Arial", 18));
    timerLabel.setStyle("-fx-text-fill: white;");

    infoRow.getChildren().addAll(scoreLabel, timerLabel);
    header.getChildren().addAll(titleLabel, infoRow);

```

```

// 1.3.9.0 The JavaFX UI shall render the Board using updateBoardUI.
boardGrid = new GridPane();
boardGrid.setAlignment(Pos.CENTER);
boardCells = new StackPane[Board.SIZE][Board.SIZE];
initializeBoardUI();

// Rack displays available tiles and supports drag input.
rackBox = new HBox(10);
rackBox.setAlignment(Pos.CENTER);

Button submitButton = new Button("Submit Turn");
Button cancelButton = new Button("Cancel Turn");

submitButton.setOnAction(e -> submitTurn());
cancelButton.setOnAction(e -> cancelTurn());

HBox buttonBox = new HBox(20, submitButton, cancelButton);
buttonBox.setAlignment(Pos.CENTER);

messageLabel = new Label();
messageLabel.setFont(Font.font(14));

root.getChildren().addAll(header, boardGrid, rackBox, buttonBox,
messageLabel);

primaryStage.setScene(new Scene(root, 950, 800));
primaryStage.setTitle("Scrabble");

refreshUI();
startTimer();
}

// 1.3.9.3 The JavaFX UI shall display game results on the end screen using
// showGameOverScreen.
// Displays the final win/loss result and final score.
private void showGameOverScreen() {
    if (timeline != null) {
        timeline.stop();
    }

    int finalScore = controller.getGameState().getCurrentScore();

    VBox root = new VBox(20);
    root.setAlignment(Pos.CENTER);
    root.setPadding(new Insets(30));

    GameState.GameStatus status = controller.getGameState().getGameStatus();

    String resultText = status == GameState.GameStatus.WON ? "You Won!" : "You
Lost!";

    Label gameOverLabel = new Label(resultText);
    gameOverLabel.setFont(Font.font(36));

    Label finalScoreLabel = new Label("Final Score: " + finalScore);
    finalScoreLabel.setFont(Font.font(24));

    Button restartButton = new Button("Start New Game");
    restartButton.setFont(Font.font(18));
    restartButton.setOnAction(e -> startGame());

```

```

        Button quitButton = new Button("Quit");
        quitButton.setFont(Font.font(18));
        quitButton.setOnAction(e -> primaryStage.close());

        HBox buttons = new HBox(20, restartButton, quitButton);
        buttons.setAlignment(Pos.CENTER);

        root.getChildren().addAll(gameOverLabel, finalScoreLabel, buttons);

        primaryStage.setScene(new Scene(root, 950, 800));
    }

    // Builds the board UI grid.
    private void initializeBoardUI() {
        boardGrid.getChildren().clear();

        for (int row = 0; row < Board.SIZE; row++) {
            for (int col = 0; col < Board.SIZE; col++) {
                StackPane cell = createBoardCell(row, col);
                boardCells[row][col] = cell;
                boardGrid.add(cell, col, row);
            }
        }
    }

    // 1.3.9.5 The JavaFX UI shall collect placement input by dropping tiles onto
    // board squares.
    // Creates a board cell and handles drag/drop placement input.
    private StackPane createBoardCell(int row, int col) {
        StackPane cell = new StackPane();
        cell.setPrefSize(CELL_SIZE, CELL_SIZE);
        cell.setStyle(getBaseCellStyle(row, col));

        cell.setOnDragOver(event -> {
            if (selectedTileUI != null &&
                controller.getGameState().getGameStatus() == GameState.GameStatus.RUNNING) {
                event.acceptTransferModes(TransferMode.MOVE);
            }
            event.consume();
        });

        cell.setOnDragDropped(event -> {
            boolean success = false;

            if (selectedTileUI != null &&
                controller.getGameState().getBoard().isValidPosition(row, col)
                && controller.getGameState().getBoard().isEmptyAt(row,
                    col) && !isPendingAt(row, col)
                && controller.getGameState().getGameStatus() ==
                GameState.GameStatus.RUNNING) {
                // 1.3.9.6 The JavaFX UI shall store unsubmitted placements
                // pendingTiles until Submit Turn is pressed with blank tile
                // handling.
                Tile tileToPlace = selectedTileUI;

                if (tileToPlace.isBlank()) {
                    TextInputDialog dialog = new TextInputDialog();
                    dialog.setTitle("Blank Tile");

```

```

        dialog.setHeaderText("Choose a letter for the blank
tile");

        dialog.setContentText("Letter:");

        String input = dialog.showAndWait().orElse("");

        if (input.length() != 1 ||
!Character.isLetter(input.charAt(0))) {
            event.setDropCompleted(false);
            event.consume();
            return;
        }

        tileToPlace =
tileToPlace.asBlankLetter(input.charAt(0));
    }

    pendingTiles.add(new PlacedTile(row, col, tileToPlace));

    selectedTileUI = null;
    success = true;
    messageLabel.setText("");
    refreshUI();
}

event.setDropCompleted(success);
event.consume();
});

return cell;
}

// Refreshes score, timer, board, and rack display from the controller/state.
private void refreshUI() {
    GameState state = controller.getGameState();

    scoreLabel.setText("Score: " + state.getCurrentScore());
    timerLabel.setText(formatTime(state.getRemainingTime()));

    updateBoardUI();
    updateRackUI();
}

// 1.3.9.0 The JavaFX UI shall render the Board using updateBoardUI.
// Updates board visuals for committed tiles, pending tiles, and bonus squares.
private void updateBoardUI() {
    for (int row = 0; row < Board.SIZE; row++) {
        for (int col = 0; col < Board.SIZE; col++) {
            StackPane cell = boardCells[row][col];
            cell.getChildren().clear();
            cell.setStyle(getBaseCellStyle(row, col));

            char committedLetter =
controller.getGameState().getBoard().getCell(row, col);
            Tile pendingTile = getPendingTileAt(row, col);

            if (committedLetter != Board.EMPTY_CELL) {
                Label tileLabel = new
Label(String.valueOf(committedLetter));
                tileLabel.setFont(Font.font(16));
                cell.getChildren().add(tileLabel);
            }
        }
    }
}

```

```

        cell.setStyle("-fx-border-color: black; -fx-background-
color: wheat;");
        } else if (pendingTile != null) {
            Label tileLabel = new
Label(String.valueOf(pendingTile.getLetter()));
            tileLabel.setFont(Font.font(16));
            cell.getChildren().add(tileLabel);
            cell.setStyle("-fx-border-color: black; -fx-background-
color: khaki;");
        } else {
            String squareText = getSquareText(row, col);
            if (!squareText.isEmpty()) {
                Label bonusLabel = new Label(squareText);
                bonusLabel.setFont(Font.font(10));

                if (squareText.equals("TW") ||
squareText.equals("DW")) {
                    bonusLabel.setStyle("-fx-text-fill:
white; -fx-font-weight: bold;");
                } else if (squareText.equals("TL") ||
squareText.equals("DL")) {
                    bonusLabel.setStyle("-fx-text-fill:
black; -fx-font-weight: bold;");
                } else if (squareText.equals("★")) {
                    bonusLabel.setStyle("-fx-text-fill:
black; -fx-font-size: 16;");
                } else {
                    bonusLabel.setStyle("-fx-text-fill:
black;");
                }

                cell.getChildren().add(bonusLabel);
            }
        }
    }
}

// 1.3.9.4 The JavaFX UI shall collect tile input through drag-and-drop from the
// rack.
// Displays rack tiles and allows the player to drag tiles onto the board.
private void updateRackUI() {
    rackBox.getChildren().clear();

    List<Tile> rackTiles = new ArrayList<>(controller.getRackTiles());

    // Hide tiles that are already pending on the board this turn.
    for (PlacedTile pending : pendingTiles) {
        rackTiles.remove(pending.getTile());
    }

    for (Tile tile : rackTiles) {
        Label tileLabel = new Label(String.valueOf(tile.getLetter()));
        tileLabel.setFont(Font.font(16));
        tileLabel.setStyle("-fx-border-color: black; -fx-background-color:
wheat; -fx-padding: 8px;");

        tileLabel.setOnDragDetected(event -> {
            if (controller.getGameState().getGameStatus() !=
GameState.GameStatus.RUNNING) {
                return;
            }
        });
    }
}

```

```

        }

        Dragboard db = tileLabel.startDragAndDrop(TransferMode.MOVE);
        ClipboardContent content = new ClipboardContent();
        content.putString(String.valueOf(tile.getLetter()));
        db.setContent(content);

        selectedTileUI = tile;
        event.consume();
    });

    rackBox.getChildren().add(tileLabel);
}

// 1.3.9.7 The JavaFX UI shall submit pending placements to GameController using
// submitTurn.
// Sends pending tiles to GameController for validation, scoring, and state
// update.
private void submitTurn() {
    if (pendingTiles.isEmpty()) {
        messageLabel.setText("No tiles placed.");
        return;
    }

    boolean success = controller.submitTurn(new ArrayList<>(pendingTiles));

    if (success) {
        pendingTiles.clear();
        selectedTileUI = null;
        messageLabel.setText("Move accepted.");
    } else {
        messageLabel.setText("Invalid move.");
    }

    refreshUI();

    if (controller.getGameState().getGameStatus() !=
GameState.GameStatus.RUNNING) {
        showGameOverScreen();
    }
}

// 1.3.9.8 The JavaFX UI shall cancel unsubmitted placements using cancelTurn
// without changing the committed Board.
// Clears pending placements and restores the UI to the last committed board
// state.
private void cancelTurn() {
    pendingTiles.clear();
    selectedTileUI = null;
    messageLabel.setText("Turn canceled.");
    refreshUI();
}

// Runs the UI timer and delegates time updates to GameController.
private void startTimer() {
    if (timeline != null) {
        timeline.stop();
    }

    timeline = new Timeline(new KeyFrame(Duration.seconds(1), event -> {

```



```

        controller.tickClock();
        refreshUI();

        if (controller.getGameState().getGameStatus() ==
GameState.GameStatus.LOST) {
            showGameOverScreen();
        }
    }));

    timeline.setCycleCount(Timeline.INDEFINITE);
    timeline.play();
}

// Checks whether a board location already contains a pending tile.
private boolean isPendingAt(int row, int col) {
    return getPendingTileAt(row, col) != null;
}

// Returns the pending tile at a board location, if one exists.
private Tile getPendingTileAt(int row, int col) {
    for (PlacedTile placed : pendingTiles) {
        if (placed.getRow() == row && placed.getCol() == col) {
            return placed.getTile();
        }
    }
    return null;
}

// Formats remaining time as MM:SS.
private String formatTime(int seconds) {
    int safeSeconds = Math.max(0, seconds);
    int min = safeSeconds / 60;
    int sec = safeSeconds % 60;
    return String.format("Time: %02d:%02d", min, sec);
}

// Returns the background style for a board square based on its bonus type.
private String getBaseCellStyle(int row, int col) {
    Square square = controller.getGameState().getBoard().getSquare(row, col);

    if (!square.isBonusUsed()) {
        switch (square.getBonusType()) {
            case DOUBLE_WORD:
                return "-fx-border-color: black; -fx-background-color: pink;";
            case TRIPLE_WORD:
                return "-fx-border-color: black; -fx-background-color: red;";
            case DOUBLE_LETTER:
                return "-fx-border-color: black; -fx-background-color:
lightblue;";
            case TRIPLE_LETTER:
                return "-fx-border-color: black; -fx-background-color:
deepskyblue;";
            case TIME:
                return "-fx-border-color: black; -fx-background-color:
lightgreen;";
            default:
                break;
        }
    }
}

```

```

        if (row == 7 && col == 7 &&
controller.getState().getBoard().isEmptyAt(row, col)) {
            return "-fx-border-color: black; -fx-background-color: gold;";
        }

        return "-fx-border-color: black; -fx-background-color: beige;";
    }

    // Returns the label text displayed on bonus squares.
    private String getSquareText(int row, int col) {
        Square square = controller.getState().getBoard().getSquare(row, col);

        if (!square.isBonusUsed()) {
            switch (square.getBonusType()) {
                case DOUBLE_WORD:
                    return row == 7 && col == 7 ? "★" : "DW";
                case TRIPLE_WORD:
                    return "TW";
                case DOUBLE_LETTER:
                    return "DL";
                case TRIPLE_LETTER:
                    return "TL";
                case TIME:
                    return "T";
                default:
                    break;
            }
        }

        return "";
    }
}

```

Main.java

```
package csc478.group2;

// Class Description: Main method class. Should only launch JavaFX.

public class Main {
    public static void main(String[] args) {
        JavaFX.launch(JavaFX.class, args);
    }
}
```

4 Testing Documentation

Test Case #	Requirement Tested / Type	Rationale	Input	Expected Output	Pass/Fail
01	1.3.1 White-box	Verify GameState stores score, time, target score, board, and status.	Create new GameState object.	Score = 0, time = 300, target = 150, status = RUNNING.	PASS
02	1.3.2 White-box	Verify board initializes as 15x15 with Square objects.	Create new Board.	All positions contain non-null Square objects.	PASS
03	1.3.3 White-box	Verify tile stores letter and point value.	Create new Tile('A', 1).	Letter is A, point value is 1.	PASS
04	1.3.4 Black-box	Verify invalid words are rejected.	Place tiles forming ZZZ and Z?Z.	Move rejected.	PASS
05	1.3.4 Black-box	Verify split placement around existing tiles works correctly.	Existing vertical word CAR; place C and Y across existing R to form CRY.	Move accepted if CRY and cross-words are valid.	PASS
06	1.3.4 Black-box	Verify invalid split placement is rejected.	Existing vertical word CAR; place C and Z across existing R to form CRZ.	Move rejected with no game state update.	PASS
07	1.3.5 White-box	Verify score includes existing tiles in formed words.	Existing R; place E to form RE.	Score calculated for full word RE, not just new tile.	PASS
08	1.3.5 White-box	Verify bingo scoring.	Place all 7 rack tiles in one valid turn, forming PEANUTS.	Score includes normal score + 50 bonus.	PASS

09	1.3.6 White-box	Verify timer decrements only while game is running.	Call decrement() while status is RUNNING.	Remaining time decreases by 1.	PASS
10	1.3.7 White-box	Verify loss condition.	Set remaining time to 0 and evaluate.	Game status becomes LOST.	PASS
11	1.3.7 White-box	Verify win condition.	Set score $\geq$ target score and evaluate.	Game status becomes WON.	PASS
12	1.3.8 White-box	Verify controller coordinates validation, scoring, rack refill, and status update.	Submit a valid move.	Board updates, score increases, rack refills, game status evaluated.	PASS
13	1.3.9 White-box	Verify UI supports drag-and-drop placement.	Drag tile from rack to empty board square.	Tile appears as pending placement on board.	PASS
14	1.3.9 White-box	Verify cancel does not modify committed board.	Drag tile to board, then click Cancel Turn.	Pending tile disappears; board returns to previous state.	PASS
15	1.3.9 Black-Box	Verify game over screen displays result.	Let timer reach 0.	End screen displays “You Lost!” and final score.	PASS
16	1.3.10 White-box	Verify logic classes can be tested independently from UI.	Instantiate and call each class without JavaFX.	Each class returns deterministic results.	PASS

5 Known Bugs & Issues

No known bugs or issues to report.